HTML • CSS • JavaScript

154 Interview Questions & Answers with Examples
(Hinglish + English)

HTML — 50 Qs | CSS — 50 Qs | JavaScript — 54 Qs

Prepared for interview & concept revision • Complete with definitions, explanations & code examples

Table of Contents

1. HTML Questions

Q1 – Q50 • HyperText Markup Language

HTML Basics

Q1. HTML kya hai? (What is HTML?)

HTML (HyperText Markup Language) ek markup language hai jo web pages ki structure banane ke liye use hoti hai. Yeh programming language nahi hai, balki tags ke through content ko define karti hai — jaise headings, paragraphs, images, links etc. Browser HTML ko parse karke usse visually render karta hai.

Q2. HTML ka full form kya hai?

HTML ka full form hai **HyperText Markup Language**. 'HyperText' ka matlab hai text jisme links (hyperlinks) hote hain jo doosre documents se connect karte hain, aur 'Markup Language' ka matlab hai tags use karke content ko structure dena.

Q3. HTML document ka basic structure kya hota hai?

Har HTML document ek basic skeleton follow karta hai jisme <!DOCTYPE html>, <html>, <head> aur <body> tags hote hain. <head> me metadata (title, links, meta tags) hota hai aur <body> me actual visible content hota hai.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
</head>
<body>
  <h1>Hello World</h1>
</body>
</html>
```

Q4. DOCTYPE declaration kyun zaroori hai?

<!DOCTYPE html> browser ko batata hai ki document HTML5 standard follow karta hai. Iske bina browser 'quirks mode' me chala jata hai jaha rendering inconsistent ho sakti hai. Isliye har HTML file ki first line yahi honi chahiye.

Q5. HTML tags aur elements me kya difference hai?

Tag woh markup hota hai jo angle brackets me likha jata hai, jaise ya . Element opening tag, content aur closing tag ka combination hota hai — jaise Hello ek complete element hai.

Q6. Void (self-closing) elements kya hote hain? Example do.

Void elements woh tags hote hain jinme closing tag ki zaroorat nahi hoti kyunki unke andar content nahi hota. Examples:
, <img>, <input>, <hr>, <meta>, <link>.

```
<img src="photo.jpg" alt="photo">
<br>
<input type="text">
```

Q7. HTML comments kaise likhte hain aur inka use kya hai?

Comments <!-- comment --> syntax me likhe jaate hain. Yeh browser me render nahi hote, sirf code ko explain karne ya temporarily code disable karne ke liye use hote hain, jisse readability improve hoti hai.

```
<!-- Yeh navbar section hai -->
<nav>...</nav>
```

Tags & Elements

Q8. Heading tags kaunse hote hain aur inka importance kya hai?

<h1> se <h6> tak 6 heading levels hote hain, <h1> sabse important (largest) aur <h6> least important (smallest) hota hai. SEO aur accessibility ke liye proper heading hierarchy maintain karna zaroori hota hai — ek page me generally ek hi <h1> hona chahiye.

Q9. aur me kya difference hai?

 ek **block-level** element hai jo apni pehle poori line le leta hai aur generally bade sections group karne ke liye use hota hai. ek **inline** element hai jo sirf content jitni jagah leta hai aur text ke chhote parts style karne ke liye use hota hai.

```
<div class="box">Block content</div>
<span class="highlight">Inline text</span>
```

Q10. Block-level aur inline elements me kya farak hai?

Block-level elements (, <p>, <h1>) new line se start hote hain aur full width lete hain. Inline elements (, <a>,) sirf content ke barabar width lete hain aur same line me continue rehte hain.

Q11. Anchor tag <a> kaise use hota hai? Attributes bhi batao.

<a> tag hyperlinks banane ke liye use hota hai. 'href' attribute destination URL define karta hai, 'target="_blank"' link ko new tab me kholta hai aur 'rel="noopener noreferrer"' security ke liye use hota hai.

```
<a href="https://example.com" target="_blank" rel="noopener">Visit Site</a>
```

Q12. Ordered aur unordered list me kya difference hai?

 (ordered list) numbered items dikhata hai jabki <ul> (unordered list) bulleted items dikhata hai. Dono ke andar list items <li> tag se define hote hain.

```
<ol><li>Step 1</li><li>Step 2</li></ol>
<ul><li>Apple</li><li>Mango</li></ul>
```

Q13. aur <b> me kya difference hai (semantic vs presentational)?

 semantic meaning deta hai — yeh batata hai ki content important hai, aur screen readers isko emphasis ke sath read karte hain. <b> sirf visually bold banata hai, koi semantic meaning nahi deta.

Q14. HTML entities kya hote hain? Example do.

HTML entities special characters represent karne ke liye use hote hain jo directly HTML syntax me conflict create karte hain, jaise < ke liye < symbol, ya & ke liye & symbol.

```
&lt;  →  <
&gt;  →  >
&amp;copy;  →  ©
&amp;nbsp;  →  non-breaking space
```

Attributes

Q15. HTML attribute kya hota hai?

Attribute ek element ko additional information provide karta hai. Yeh opening tag ke andar `name="value"` format me likha jata hai, jaise src, href, class, id etc.

```
<img src="cat.jpg" alt="A cat" width="200">
```

Q16. class aur id attribute me kya difference hai?

'id' unique hota hai — ek page me sirf ek element ke paas woh particular id ho sakti hai, aur CSS/JS me single element target karne ke liye use hoti hai. 'class' reusable hota hai — multiple elements same class share kar sakte hain, group styling ke liye use hoti hai.

```
<div id="header">...</div>
<p class="highlight">Text</p>
<p class="highlight">More text</p>
```

Q17. data-* custom attributes kya hote hain?

data-* attributes custom data store karne ke liye use hote hain jo JavaScript se access kiye ja sakte hain, bina koi non-standard attribute use kiye. Yeh dataset property se access hote hain.

```
<div id="user" data-user-id="42"></div>

<script>
  console.log(document.getElementById("user").dataset.userId); // "42"
</script>
```

Q18. alt attribute kyun important hai images ke liye?

alt attribute image ka text description deta hai jo image load na hone par ya screen readers (accessibility) ke liye display hota hai. Yeh SEO ke liye bhi helpful hai.

Q19. target attribute ki different values kya hoti hain?

target attribute link kaise open ho yeh define karta hai: `_self` (same tab, default), `_blank` (new tab), `_parent` (parent frame), `_top` (full window).

Forms

Q20. HTML form kya hota hai aur uske main elements kya hain?

<form> user input collect karne aur server ko submit karne ke liye use hota hai. Main elements: <input>, <textarea>, <select>, <button>, <label>.

```
<form action="/submit" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name">
  <button type="submit">Submit</button>
</form>
```

Q21. GET aur POST method me form submission ka kya difference hai?

GET method data ko URL query string me bhejta hai (visible, limited size, bookmarkable) — searching ke liye acha hai. POST method data ko request body me bhejta hai (hidden, unlimited size) — sensitive data ya large data submit karne ke liye use hota hai.

Q22. input type attribute ki kuch important values batao.

text, password, email, number, date, checkbox, radio, file, submit, tel, url, range — har type browser ko different validation aur UI provide karne me help karta hai.

```
<input type="email" required>
<input type="date">
<input type="checkbox" checked>
```

Q23. <label> tag ka use kya hai aur 'for' attribute kyun zaroori hai?

<label> input field ke sath text caption jodta hai. 'for' attribute label ko input ke 'id' se link karta hai — isse click karne par bhi input focus ho jata hai, jo accessibility ke liye bahut important hai.

```
<label for="email">Email</label>
<input type="email" id="email">
```

Q24. required, placeholder aur disabled attributes kya karte hain?

'required' field ko mandatory banata hai (form submit nahi hoga khali hone par). 'placeholder' hint text dikhata hai jo type karte hi gayab ho jata hai. 'disabled' field ko non-interactive aur greyed out kar deta hai.

```
<input type="text" placeholder="Enter name" required>
<input type="text" disabled>
```

Q25. HTML5 form validation kaise kaam karti hai?

HTML5 built-in attributes jaise required, pattern, min, max, type="email" ke through automatic client-side validation provide karta hai — bina JavaScript likhe browser khud invalid input par error message dikhata hai.

```
<input type="text" pattern="[A-Za-z]{3,}" title="Minimum 3 letters">
```

Q26. <select> aur <option> kaise use hote hain?

<select> ek dropdown list banata hai jisme multiple <option> tags hote hain jinme se user ek (ya multiple, agar multiple attribute ho) choose kar sakta hai.

```
<select name="fruit">
  <option value="apple">Apple</option>
  <option value="mango" selected>Mango</option>
</select>
```

Tables

Q27. HTML table kaise banate hain? Main tags batao.

<table> parent element hai. <tr> row define karta hai, <th> header cell aur <td> data cell banata hai.

```
<table>
  <tr><th>Name</th><th>Age</th></tr>
  <tr><td>Nik</td><td>21</td></tr>
</table>
```

Q28. <thead>, <tbody> aur <tfoot> ka use kya hai?

Yeh semantic table sections hain — <thead> header rows group karta hai, <tbody> main data rows hold karta hai, aur <tfoot> footer/summary rows ke liye hota hai. Yeh styling aur accessibility dono behtar karta hai.

Q29. colspan aur rowspan attributes kya karte hain?

'colspan' ek cell ko multiple columns tak fail (merge) karne deta hai, aur 'rowspan' cell ko multiple rows tak fail karne deta hai.

```
<td colspan="2">Merged across 2 columns</td>
<td rowspan="2">Merged across 2 rows</td>
```

Q30. Table caption kaise add karte hain?

<caption> tag table ke andar sabse pehle likha jata hai aur table ka title/description dikhata hai, jo accessibility ke liye bhi useful hota hai.

```
<table>
  <caption>Student Marks</caption>
  ...
</table>
```

Semantic HTML

Q31. Semantic HTML kya hota hai aur iska fayda kya hai?

Semantic elements woh tags hain jo apne content ka meaning batate hain — jaise <header>, <article>. Inse code readability, SEO aur accessibility (screen readers) sab improve hote hain, compared to sirf use karne se.

Q32. <header>, <footer>, <nav> kab use karte hain?

<header> page/section ka introductory content (logo, title) hold karta hai. <footer> bottom content (copyright, links) ke liye hota hai. <nav> navigation links group karne ke liye use hota hai.

```
<header><h1>My Site</h1></header>
<nav><a href="/">Home</a></nav>
<footer>&copy; 2026</footer>
```

Q33. <article> aur <section> me kya difference hai?

<article> independently distributable content hota hai (jaise blog post, news item) jo standalone sense banata hai. <section> ek page ke thematic grouping ke liye hota hai jo generally heading ke sath aata hai, lekin standalone nahi hota.

Q34. <main> tag ka kya purpose hai?

<main> page ka primary, unique content wrap karta hai (header, footer, sidebar exclude karke). Ek page me sirf ek <main> hona chahiye, aur yeh accessibility tools ko main content jaldi identify karne me madad karta hai.

Q35. <figure> aur <figcaption> kya karte hain?

<figure> self-contained content (image, diagram, code) ko wrap karta hai, aur <figcaption> uska caption/description provide karta hai.

```
<figure>
  <img src="chart.png" alt="Sales chart">
  <figcaption>Fig 1: Monthly Sales</figcaption>
</figure>
```

Q36. <aside> tag kab use karte hain?

<aside> main content se related lekin indirectly related content ke liye use hota hai — jaise sidebar, pull quotes, ya related links.

Media (Images, Audio, Video)

Q37. <img> tag ke important attributes kya hain?

src (image path), alt (alternate text), width/height (dimensions), loading="lazy" (performance ke liye lazy loading).

```
<img src="banner.jpg" alt="Banner" width="600" loading="lazy">
```

Q38. HTML5 me video kaise embed karte hain?

<video> tag use hota hai controls, autoplay, loop, muted jaise attributes ke sath. Multiple <source> tags different formats provide karne ke liye use hote hain (browser compatibility).

```
<video controls width="400">
  <source src="movie.mp4" type="video/mp4">
  Your browser does not support video.
</video>
```

Q39. HTML5 me audio kaise embed karte hain?

<audio> tag controls attribute ke sath use hota hai, jisme <source> tag audio file ka path deta hai.

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
</audio>
```

Q40. <picture> element kya hai aur kab use karte hain?

<picture> responsive images ke liye use hota hai — different screen sizes ya formats ke liye alag alag image sources define kar sakte hain via multiple <source> tags, browser sabse suitable choose karta hai.

```
<picture>
  <source media="(min-width:600px)" srcset="large.jpg">
  <img src="small.jpg" alt="Responsive">
</picture>
```

Q41. srcset attribute ka use kya hai?

srcset ek image ke multiple resolutions define karne deta hai taaki browser device ke screen density/size ke hisaab se best image load kare, jisse performance improve hoti hai.

```
<img src="small.jpg" srcset="medium.jpg 1000w, large.jpg 2000w" alt="Demo">
```

Meta Tags & SEO

Q42. <meta> tag ka use kya hai?

<meta> tag page ke baare me metadata provide karta hai jo browser aur search engines ke liye hota hai — jaise character encoding, viewport settings, description, keywords.

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Q43. viewport meta tag kyun zaroori hai (responsive design)?

Yeh mobile devices ko batata hai page ko device ki width ke hisaab se render kare, jo responsive design ke liye essential hai. Bina iske mobile browsers page ko desktop width me render karte hain aur zoom out kar dete hain.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Q44. meta description tag SEO me kaise help karta hai?

meta description search engine results (SERP) me page ke title ke neeche summary ke roop me dikhta hai, jo click-through rate improve karne me help karta hai — yeh direct ranking factor nahi hai lekin indirect impact hota hai.

```
<meta name="description" content="Learn HTML CSS JS with examples">
```

Q45. <link> tag ka use kya hota hai?

<link> external resources ko HTML se connect karta hai, jaise CSS stylesheets, favicons, ya preload/prefetch hints.

```
<link rel="stylesheet" href="style.css">
<link rel="icon" href="favicon.ico">
```

Miscellaneous / Advanced HTML

Q46. iframe kya hota hai aur kab use karte hain?

<iframe> ek HTML page ke andar doosra HTML document embed karne ke liye use hota hai — jaise YouTube video, Google map embed karna. Security ke liye sandbox attribute use kiya jata hai.

```
<iframe src="https://example.com" width="600" height="400"></iframe>
```

Q47. HTML5 me localStorage aur sessionStorage kya hote hain?

Yeh Web Storage APIs hain jo browser me key-value data store karte hain. localStorage data permanently (page close hone ke baad bhi) store karta hai, jabki sessionStorage sirf tab open hone tak data rakhta hai.

```
localStorage.setItem("name", "Nik");
console.log(localStorage.getItem("name")); // "Nik"
```

Q48. <canvas> tag ka use kya hai?

<canvas> ek drawable region provide karta hai jisme JavaScript (Canvas API) ke through graphics, charts, games ya animations dynamically draw kiye ja sakte hain.

```
<canvas id="myCanvas" width="200" height="100"></canvas>
```

Q49. HTML5 se pehle aur baad me kya major changes aaye?

HTML5 me naye semantic tags (header, footer, article), multimedia support (audio, video bina plugins ke), canvas/SVG graphics, form input types, aur APIs jaise geolocation, localStorage add hue.

Q50. HTML validation kya hai aur kyun important hai?

HTML validation check karta hai ki code W3C standards follow karta hai ya nahi (tools jaise W3C Validator se). Valid HTML cross-browser compatibility, SEO aur maintainability improve karta hai.

2. CSS Questions

Q51 – Q100 • Cascading Style Sheets

CSS Basics

Q51. CSS kya hai?

CSS (Cascading Style Sheets) ek styling language hai jo HTML elements ki presentation (colors, layout, fonts, spacing) control karti hai. Yeh content (HTML) ko design (visual style) se separate karti hai.

Q52. CSS ko HTML me include karne ke teen tareeke kya hain?

1) **Inline CSS** - style attribute directly element par. 2) **Internal CSS** - <style> tag <head> me. 3) **External CSS** - alag .css file link tag se link ki jaati hai (best practice, reusable aur maintainable).

```
<p style="color:red;">Inline</p>

<style>
  p { color: blue; }
</style>

<link rel="stylesheet" href="style.css">
```

Q53. CSS syntax kya hoti hai?

CSS rule ek selector aur declaration block se milkar banta hai. Declaration block me property:value pairs semicolon se separate hoke curly braces ke andar likhe jate hain.

```
selector {
  property: value;
  property2: value2;
}
```

Q54. CSS specificity kya hai?

Specificity ek algorithm hai jo browser ko decide karne me help karta hai konsa CSS rule apply hoga jab multiple rules same element ko target karte hain. Order (highest to lowest): inline styles > IDs > classes/attributes/pseudo-classes > elements/pseudo-elements.

Q55. !important ka use kya hai aur kyun avoid karna chahiye?

!important ek declaration ki priority sabse high bana deta hai, normal specificity rules ko override karke. Isse avoid karna chahiye kyunki yeh debugging mushkil bana deta hai aur CSS cascade ko break kar deta hai — sirf emergency cases me use karna chahiye.

```
p { color: red !important; }
```

Q56. CSS comments kaise likhte hain?

CSS me comments `/* comment */` syntax me likhe jate hain. Yeh browser ignore kar deta hai.

```
/* Yeh header ki styling hai */
header { background: #333; }
```

Selectors

Q57. CSS selectors ke main types kaunse hain?

Element selector (p), class selector (.name), ID selector (#name), universal selector (*), attribute selector ([type="text"]), aur combinators (descendant, child, sibling).

```
p { }
.box { }
#header { }
* { }
[type='text'] { }
```

Q58. Descendant aur child selector me kya difference hai?

Descendant selector (`div p`) us element ke andar sabhi nested elements ko select karta hai, kitne bhi levels deep ho. Child selector (`div > p`) sirf directly ander wale (immediate child) elements ko select karta hai.

```
div p { color: red; }      /* all nested p */
div > p { color: blue; }   /* only direct children */
```

Q59. Pseudo-class kya hota hai? Examples do.

Pseudo-class ek element ki special state define karta hai. Examples: `:hover` (mouse over), `:focus` (focused input), `:nth-child()`, `:first-child`, `:disabled`.

```
button:hover { background: green; }
input:focus { border-color: blue; }
```

Q60. Pseudo-element kya hota hai? Examples do.

Pseudo-element ek element ke specific part ko style karta hai jo actual DOM me nahi hota. Examples: `::before`, `::after`, `::first-line`, `::placeholder`. Double colon (::) modern syntax hai.

```
.box::before {
  content: "★ ";
  color: gold;
}
```

Q61. Attribute selector kaise use karte hain?

Attribute selector elements ko unke attribute value ke basis par select karta hai — jaise `[type="text"]` sabhi text input select karega.

```
input[type='text'] { border: 1px solid gray; }
a[target='_blank'] { color: purple; }
```

Q62. Sibling selectors (+ aur ~) me kya difference hai?

+ (adjacent sibling) sirf immediately next element ko select karta hai. ~ (general sibling) us element ke baad wale sabhi matching siblings ko select karta hai.

```
h1 + p { }   /* immediately after h1 */
h1 ~ p { }   /* all p after h1 */
```

Q63. Grouping selector kya hota hai?

Multiple selectors ko comma se separate karke ek hi style declaration di ja sakti hai, isse code repeat nahi karna padta.

```
h1, h2, h3 {
  font-family: Arial, sans-serif;
}
```

Box Model

Q64. CSS Box Model kya hai?

Har HTML element ek rectangular box hota hai jisme 4 parts hote hain (andar se bahar): **Content**, **Padding** (content ke around space), **Border**, aur **Margin** (element ke bahar space).

Q65. box-sizing property ka use kya hai?

box-sizing decide karta hai width/height calculation me padding-border include ho ya nahi. `content-box` (default) me width sirf content ke liye hoti hai. `border-box` me width me padding aur border bhi shamil hote hain — isse layout predict karna easy hota hai.

```
* {
  box-sizing: border-box;
}
```

Q66. margin aur padding me kya difference hai?

Padding element ke content aur border ke beech ka space hota hai (element ke andar). Margin element ke border se bahar doosre elements tak ka space hota hai (element ke bahar).

```
.box {
  padding: 20px;   /* inside spacing */
  margin: 10px;    /* outside spacing */
}
```

Q67. Margin collapsing kya hai?

Jab do vertical margins (jaise ek element ka bottom margin aur next element ka top margin) touch karte hain, to woh add nahi hote balki bade wale margin ka value final hota hai — isko margin collapsing kehte hain.

Q68. border-radius property kya karti hai?

border-radius element ke corners ko round karti hai, jisse rounded boxes ya circles bante hain.

```
.card {
  border-radius: 12px;
}
.circle {
  border-radius: 50%;
}
```

Q69. outline aur border me kya difference hai?

Border box model ka part hai aur space occupy karta hai (layout affect karta hai). Outline border ke bahar draw hota hai aur layout space nahi leta, generally focus indication ke liye use hota hai.

Positioning & Display

Q70. CSS display property ki values kya kya hain?

block (full width, new line), inline (content-width, same line), inline-block (inline + width/height settable), none (hide element), flex, grid.

```
.box { display: flex; }
.hidden { display: none; }
```

Q71. position property ki values samjhao (static, relative, absolute, fixed, sticky).

static default normal flow. **relative** apni normal position ke relative shift hota hai. **absolute** nearest positioned ancestor ke relative position hota hai (normal flow se hat jata hai). **fixed** viewport ke relative fixed rehta hai (scroll par bhi). **sticky** scroll tak relative, phir fixed ban jata hai.

```
.navbar {
  position: sticky;
  top: 0;
}
```

Q72. z-index kya karta hai?

z-index overlapping elements ka stacking order decide karta hai — higher value wala element upar dikhta hai. Yeh sirf positioned elements (relative, absolute, fixed, sticky) par kaam karta hai.

```
.modal { position: fixed; z-index: 1000; }
```

Q73. CSS float property kya hai aur ab kyun kam use hoti hai?

float element ko left ya right push karke text ko uske around wrap karne deta hai — pehle layouts banane ke liye use hota tha. Ab Flexbox aur Grid ke aane se float layouts ke liye deprecated approach maana jata hai.

```
.img { float: left; margin-right: 10px; }
```

Q74. visibility:hidden aur display:none me kya difference hai?

display:none element ko completely remove kar deta hai (space bhi nahi leta, layout reflow hota hai). visibility:hidden element ko invisible banata hai lekin uski space page me reserved rehti hai.

Q75. overflow property kya karti hai?

overflow define karta hai jab content container se bada ho jaye to kya ho: `visible` (default, overflow ho jata hai), `hidden` (cut ho jata hai), `scroll` (scrollbar aata hai), `auto` (zaroorat par scrollbar).

```
.box { overflow: auto; height: 100px; }
```

Flexbox

Q76. Flexbox kya hai aur kab use karna chahiye?

Flexbox ek one-dimensional layout model hai jo items ko row ya column me easily align/distribute karne deta hai. Yeh navbars, card alignments, ya kisi bhi single-direction layout ke liye best hai.

```
.container {
  display: flex;
}
```

Q77. justify-content aur align-items me kya difference hai?

justify-content items ko **main axis** (default: horizontal) par align karta hai. align-items items ko **cross axis** (default: vertical) par align karta hai.

```
.container {
  display: flex;
  justify-content: center; /* horizontal */
  align-items: center;     /* vertical */
}
```

Q78. flex-direction property kya karti hai?

flex-direction decide karta hai items kis direction me arrange honge: `row` (default, horizontal), `column` (vertical), `row-reverse`, `column-reverse`.

```
.container { flex-direction: column; }
```

Q79. flex-grow, flex-shrink, flex-basis kya karte hain?

flex-grow item ko available extra space lene deta hai (proportion me). **flex-shrink** jab space kam ho to item ko shrink hone deta hai. **flex-basis** item ki initial default size set karta hai (before growing/shrinking).

```
.item {
  flex-grow: 1;
  flex-shrink: 1;
  flex-basis: 200px;
}
/* shorthand: flex: 1 1 200px; */
```

Q80. flex-wrap property ka use kya hai?

flex-wrap decide karta hai jab items container me fit na ho to unhe next line par wrap kiya jaye ya nahi. Default nowrap hota hai (single line, shrink kar dete hain), wrap multiple lines allow karta hai.

```
.container { flex-wrap: wrap; }
```

Q81. gap property flexbox/grid me kya karti hai?

gap items ke beech ka spacing set karti hai bina margin use kiye — row-gap aur column-gap alag se ya shorthand gap se define kiye ja sakte hain.

```
.container { display: flex; gap: 16px; }
```

Grid

Q82. CSS Grid kya hai aur Flexbox se kaise different hai?

CSS Grid ek two-dimensional layout system hai jo rows aur columns dono simultaneously control karta hai. Flexbox one-dimensional hota hai (ek time par ya row ya column). Complex page layouts ke liye Grid better hai.

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

Q83. grid-template-columns aur grid-template-rows kya karte hain?

Yeh properties define karti hain ki grid me kitne columns/rows honge aur unki size kya hogi (px, %, fr unit me).

```
.container {
  grid-template-columns: 200px 1fr 1fr;
  grid-template-rows: 100px auto;
}
```

Q84. CSS Grid me 'fr' unit ka matlab kya hai?

'fr' (fractional unit) available space ka proportion represent karta hai. Jaise `1fr 2fr` ka matlab hai second column pehle wale se double space lega.

```
grid-template-columns: 1fr 2fr; /* col2 is twice col1 */
```

Q85. grid-column aur grid-row se item ka span kaise control karte hain?

Yeh properties ek grid item ko specific columns/rows tak span karne deta hai using start/end line numbers.

```
.item {
  grid-column: 1 / 3;  /* spans column 1 to 3 */
  grid-row: 1 / 2;
}
```

Q86. grid-template-areas ka use kya hai?

grid-template-areas named layout areas define karne deta hai text-based visual syntax me, jisse complex layouts (header, sidebar, main, footer) intuitively define kiye ja sakte hain.

```
.container {
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
}
.header { grid-area: header; }
```

Responsive Design / Media Queries

Q87. Responsive web design kya hota hai?

Responsive design woh approach hai jisme website different screen sizes (mobile, tablet, desktop) par automatically adapt hoti hai, using flexible grids, images aur media queries.

Q88. Media query kaise likhte hain?

@media rule specific conditions (screen width, orientation) ke basis par CSS apply karta hai.

```
@media (max-width: 600px) {
  .container {
    flex-direction: column;
  }
}
```

Q89. Mobile-first approach kya hai?

Mobile-first me pehle small screens ke liye base CSS likhi jaati hai, phir `min-width` media queries se progressively larger screens ke liye styles add ki jaati hain. Yeh performance aur maintainability ke liye better maana jata hai.

```
/* base: mobile styles */
.box { width: 100%; }

@media (min-width: 768px) {
  .box { width: 50%; }
}
```

Q90. px, em, rem, % units me kya difference hai?

px fixed absolute unit hai. **em** parent element ke font-size ke relative hota hai. **rem** root (html) element ke font-size ke relative hota hai (zyada predictable). **%** parent container ke relative hota hai.

```
html { font-size: 16px; }
.box { font-size: 2rem; } /* = 32px, relative to root */
```

Q91. viewport units (vw, vh) kya hote hain?

vw aur vh viewport ki width aur height ka percentage represent karte hain. 1vw = viewport width ka 1%, 1vh = viewport height ka 1%. Full-screen sections banane ke liye useful hai.

```
.hero { height: 100vh; }
```

Animations & Transitions

Q92. CSS transition kya hai?

Transition ek property ki value change hone par smooth animation create karta hai over a specified duration, bina JavaScript use kiye.

```
.btn {
  background: blue;
  transition: background 0.3s ease;
}
.btn:hover {
  background: red;
}
```

Q93. CSS animation aur transition me kya difference hai?

Transition sirf do states (start aur end) ke beech change animate karta hai, jo trigger (hover/click) chahiye hota hai. Animation (@keyframes ke sath) multiple steps/states define kar sakta hai aur automatically, repeatedly bhi chal sakta hai bina trigger ke.

Q94. @keyframes kaise use karte hain?

@keyframes ek animation sequence define karta hai using percentage stops (0% to 100%) ya from/to keywords, phir animation property se element par apply kiya jata hai.

```
@keyframes slide {
  from { transform: translateX(0); }
  to { transform: translateX(100px); }
}
.box {
  animation: slide 2s infinite;
}
```

Q95. transform property kya karti hai? Examples do.

transform element ko visually modify karta hai without affecting layout flow — translate() (move), rotate() (ghumana), scale() (size change), skew() (tilt).

```
.box {
  transform: rotate(45deg) scale(1.2);
}
```

Q96. CSS transitions me 'ease', 'linear' jaise timing functions ka kya matlab hai?

Timing function animation ki speed curve define karta hai: `linear` (constant speed), `ease` (slow start-fast-slow end, default), `ease-in` (slow start), `ease-out` (slow end).

```
.box { transition: all 0.5s ease-in-out; }
```

Pseudo-classes, Variables & Advanced

Q97. CSS custom properties (variables) kaise use karte hain?

CSS variables `--name: value;` se define hote hain (generally :root me) aur `var(--name)` se use hote hain. Yeh reusable values aur theming (dark mode etc.) ke liye bahut useful hai.

```
:root {
  --primary-color: #3498db;
}
.btn {
  background: var(--primary-color);
}
```

Q98. :nth-child() selector kaise kaam karta hai?

:nth-child(n) formula ke basis par elements select karta hai — jaise `:nth-child(2n)` even elements, `:nth-child(odd)` odd elements select karta hai.

```
li:nth-child(odd) { background: #eee; }
li:nth-child(2) { color: red; }
```

Q99. CSS specificity calculate kaise karte hain (example ke sath)?

Specificity ek 4-value score hoti hai (inline, IDs, classes/attributes, elements). Jaise `#nav .item p` = 1 ID + 1 class + 1 element, jo `.item p` (0 ID + 1 class + 1 element) se zyada specific hai isliye win karega.

Q100. CSS me responsive typography ke liye clamp() ka use kaise hota hai?

clamp(min, preferred, max) ek value ko minimum aur maximum ke beech restrict karta hai, jo bina media query ke fluid/responsive font sizes banane deta hai.

```
h1 { font-size: clamp(1.5rem, 4vw, 3rem); }
```

3. JavaScript Questions

Q101 – Q154 • Scripting & Logic

JavaScript Basics

Q101. JavaScript kya hai?

JavaScript ek high-level, interpreted programming language hai jo web pages ko interactive aur dynamic banati hai. Yeh browser me directly run hoti hai aur DOM manipulation, events handling, aur logic execution ke liye use hoti hai.

Q102. JavaScript ko HTML me kaise include karte hain?

<script> tag use hota hai — internal (script tag ke andar code) ya external (src attribute se .js file link). Best practice hai script ko body ke end me ya defer attribute ke sath rakhna.

```
<script src="app.js" defer></script>

<script>
  console.log("Hello");
</script>
```

Q103. JavaScript ek interpreted language hai ya compiled?

JavaScript primarily interpreted language hai, lekin modern engines (jaise V8) JIT (Just-In-Time) compilation use karte hain performance improve karne ke liye — matlab code run time par machine code me convert hota hai.

Q104. console.log() ka use kya hai?

console.log() browser ki developer console me values/messages print karta hai — debugging ke liye sabse commonly used method hai.

```
console.log("Hello", 42, true);
```

Q105. == aur === operator me kya difference hai?

== (loose equality) comparison se pehle type conversion karta hai. === (strict equality) type aur value dono check karta hai bina conversion ke — isliye === use karna recommended hai.

```
5 == "5"   // true (type conversion)
5 === "5"  // false (different types)
```

Q106. undefined aur null me kya difference hai?

undefined ka matlab hai variable declare hua hai lekin value assign nahi hui. null ek intentional 'empty value' hai jo developer explicitly assign karta hai.

```
let a;
console.log(a); // undefined

let b = null;
console.log(b); // null
```

Variables & Data Types

Q107. var, let aur const me kya difference hai?

var function-scoped hota hai aur re-declare/update dono ho sakta hai. **let** block-scoped hai aur update ho sakta hai but re-declare nahi. **const** block-scoped hai aur na update na re-declare ho sakta hai (constant value).

```
var x = 1;
let y = 2;
const z = 3;
y = 20;   // ok
z = 30;   // Error
```

Q108. JavaScript ke primitive data types kaunse hain?

String, Number, Boolean, Undefined, Null, Symbol, aur BigInt — total 7 primitive types hain.

```
let str = "hello";
let num = 42;
let bool = true;
let sym = Symbol("id");
```

Q109. Primitive aur reference (non-primitive) types me kya difference hai?

Primitive types (string, number, boolean etc.) **by value** store/copy hote hain. Reference types (objects, arrays, functions) **by reference** store hote hain — variable memory address point karta hai.

```
let a = 5;
let b = a;
b = 10;
console.log(a); // 5 (unaffected)

let obj1 = {x: 1};
let obj2 = obj1;
obj2.x = 100;
console.log(obj1.x); // 100 (affected)
```

Q110. typeof operator kya karta hai?

typeof kisi variable/value ka data type string ke form me return karta hai — debugging aur type checking ke liye useful.

```
typeof "hello"   // "string"
typeof 42        // "number"
typeof true       // "boolean"
typeof undefined  // "undefined"
```

Q111. Type coercion kya hai?

Type coercion automatic ya implicit conversion hai ek data type se doosre me, jab operators different types ke operands ke sath use hote hain.

```
console.log("5" + 3);   // "53" (number converted to string)
console.log("5" - 3);   // 2 (string converted to number)
```

Q112. NaN kya hota hai?

NaN (Not-a-Number) ek special value hai jo invalid ya undefined mathematical operations ka result hota hai. Interesting fact: `typeof NaN` "number" return karta hai, aur `NaN === NaN` false hota hai.

```
console.log(0 / "abc");  // NaN
console.log(NaN === NaN); // false
console.log(isNaN(NaN));  // true
```

Q113. Template literals kya hote hain?

Template literals backticks (`) se likhe jate hain aur `${expression}` syntax se variables/expressions ko directly string ke andar embed karne dete hain, multi-line strings bhi allow karte hain.

```
let name = "Nik";
console.log(`Hello, ${name}! You are ${20+1} years old.`);
```

Functions

Q114. JavaScript function kya hai aur kaise define karte hain?

Function code ka reusable block hota hai jo specific task perform karta hai. Function declaration, function expression, ya arrow function se define kiya ja sakta hai.

```
function greet(name) {
  return `Hello ${name}`;
}
console.log(greet("Nik"));
```

Q115. Function declaration aur function expression me kya difference hai?

Function declaration hoisted hoti hai (call before define kar sakte hain). Function expression hoisted nahi hoti (variable assignment ki tarah), isliye define hone se pehle call nahi kar sakte.

```
greet(); // works (hoisted)
function greet() { console.log("Hi"); }

greet2(); // Error
const greet2 = function() { console.log("Hi"); };
```

Q116. Arrow functions kya hote hain aur normal functions se kaise different hain?

Arrow functions concise syntax provide karte hain aur apna khud ka `this` binding nahi rakhte — yeh enclosing (lexical) scope ka `this` use karte hain. Yeh constructor ke roop me use nahi ho sakte.

```
const add = (a, b) => a + b;
console.log(add(2, 3)); // 5
```

Q117. Default parameters kya hote hain?

Default parameters function ko default value assign karne dete hain agar argument pass na kiya jaye ya undefined ho.

```
function greet(name = "Guest") {
  console.log(`Hello ${name}`);
}
greet(); // Hello Guest
```

Q118. Rest parameters (...args) kya karte hain?

Rest parameter multiple arguments ko ek array me collect karta hai, jab function me indefinite number of arguments pass ho sakte hain.

```
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
console.log(sum(1, 2, 3, 4)); // 10
```

Q119. Higher-order function kya hota hai?

Higher-order function woh function hota hai jo doosre function ko argument ki tarah accept karta hai ya function return karta hai — jaise map(), filter(), reduce().

```
function multiply(fn, arr) {
  return arr.map(fn);
}
multiply(x => x*2, [1,2,3]); // [2,4,6]
```

Q120. IIFE (Immediately Invoked Function Expression) kya hai?

IIFE ek function hai jo define hone ke turant baad automatically execute ho jata hai. Isse variable scope isolated rakhne me help milti hai (global namespace pollute nahi hota).

```
(function() {
  console.log("Runs immediately!");
})();
```

Arrays & Objects

Q121. JavaScript array kya hota hai?

Array ek ordered collection hai jisme multiple values (kisi bhi type ki) index (0 se start) ke through store hoti hain.

```
let fruits = ["apple", "banana", "mango"];
console.log(fruits[0]); // "apple"
```

Q122. map(), filter() aur reduce() me kya difference hai?

map() har element par function apply karke naya array return karta hai (same length). **filter()** condition ke basis par elements select karke naya array banata hai (chhota ho sakta hai). **reduce()** array ko single value me accumulate/reduce karta hai.

```
let nums = [1,2,3,4];
nums.map(n => n*2);            // [2,4,6,8]
nums.filter(n => n%2===0);      // [2,4]
nums.reduce((a,b) => a+b, 0);   // 10
```

Q123. Array me push, pop, shift, unshift ka use kya hai?

push() end me element add karta hai. **pop()** end se element remove karta hai. **unshift()** start me element add karta hai. **shift()** start se element remove karta hai.

```
let arr = [1,2,3];
arr.push(4);   // [1,2,3,4]
arr.pop();     // [1,2,3]
arr.unshift(0);// [0,1,2,3]
arr.shift();   // [1,2,3]
```

Q124. JavaScript object kya hota hai?

Object key-value pairs ka collection hota hai jisme properties aur methods store hote hain, curly braces {} se define hota hai.

```
let person = {
  name: "Nik",
  age: 21,
  greet() { return `Hi, I am ${this.name}`; }
};
```

Q125. Object destructuring kya hai?

Destructuring object/array se values ko easily individual variables me extract karne deta hai, concise syntax me.

```
const { name, age } = { name: "Nik", age: 21 };
console.log(name, age); // Nik 21
```

Q126. Spread operator (...) kya karta hai?

Spread operator array/object ke elements ko expand (spread) karta hai — copying, merging, ya function arguments pass karne ke liye use hota hai.

```
let arr1 = [1, 2];
let arr2 = [...arr1, 3, 4]; // [1,2,3,4]

let obj1 = {a: 1};
let obj2 = {...obj1, b: 2}; // {a:1, b:2}
```

Q127. Array aur Object me for...in aur for...of me kya difference hai?

for...in object ki keys (ya array ke indices) ke over iterate karta hai. **for...of** array/iterable ke values ke over directly iterate karta hai.

```
let arr = [10, 20, 30];
for (let i in arr) console.log(i);    // 0, 1, 2 (indices)
for (let v of arr) console.log(v);    // 10, 20, 30 (values)
```

DOM Manipulation

Q128. DOM kya hota hai?

DOM (Document Object Model) HTML document ka tree-structured representation hai jo JavaScript ko page ke elements ko access aur manipulate karne deta hai.

Q129. document.getElementById() aur document.querySelector() me kya difference hai?

getElementById() sirf ID se ek element select karta hai (fast, specific). querySelector() kisi bhi CSS selector (class, id, tag, attribute) se pehla matching element return karta hai — flexible hai.

```
document.getElementById("header");
document.querySelector(".header");
document.querySelector("#header");
```

Q130. querySelector aur querySelectorAll me kya difference hai?

querySelector() sirf pehla matching element return karta hai. querySelectorAll() sabhi matching elements ki NodeList return karta hai.

```
let first = document.querySelector(".item");
let all = document.querySelectorAll(".item");
```

Q131. innerHTML aur textContent me kya difference hai?

innerHTML element ke andar HTML markup ko string ke roop me set/get karta hai (HTML parse hota hai). textContent sirf plain text set/get karta hai, koi HTML parse nahi hota (safer, XSS attacks se bachata hai).

```
el.innerHTML = "<b>Bold</b>"; // renders as bold
el.textContent = "<b>Bold</b>"; // shows literal text
```

Q132. createElement() aur appendChild() kaise kaam karte hain?

createElement() naya HTML element create karta hai (memory me, page me nahi). appendChild() us element ko kisi parent element ke andar last child ki tarah insert karta hai.

```
let div = document.createElement("div");
div.textContent = "New Box";
document.body.appendChild(div);
```

Q133. classList.add(), remove(), toggle() ka use kya hai?

classList element ki classes ko manage karta hai — add() class add karta hai, remove() hata deta hai, toggle() present hone par remove aur na hone par add kar deta hai.

```
el.classList.add("active");
el.classList.remove("hidden");
el.classList.toggle("open");
```

Events

Q134. JavaScript events kya hote hain?

Events browser me hone wale actions/occurrences hote hain (click, keypress, load, submit etc.) jinko JavaScript listen aur handle kar sakta hai.

Q135. addEventListener() kaise use karte hain?

addEventListener() ek element par event listener attach karta hai jo specified event trigger hone par callback function run karta hai. Yeh multiple listeners same event ke liye support karta hai.

```
button.addEventListener("click", function() {
  console.log("Clicked!");
});
```

Q136. Event bubbling kya hai?

Event bubbling me jab kisi child element par event trigger hota hai, to woh event upar uske parent, grandparent tak propagate (bubble up) hota hai, jab tak document tak nahi pahunch jata.

```
parent.addEventListener("click", () => console.log("Parent clicked"));
child.addEventListener("click", () => console.log("Child clicked"));
// Clicking child logs: "Child clicked" then "Parent clicked"
```

Q137. event.preventDefault() ka use kya hai?

preventDefault() event ke default browser behavior ko rokta hai — jaise form submit hone se rokna ya link navigation cancel karna.

```
form.addEventListener("submit", function(e) {
  e.preventDefault();
  console.log("Form submission stopped");
});
```

Q138. Event delegation kya hota hai?

Event delegation ek technique hai jisme parent element par ek hi event listener lagaya jata hai jo uske sabhi children ke events handle karta hai (event bubbling use karke) — dynamically added elements ke liye bhi kaam karta hai aur performance behtar hoti hai.

```
document.querySelector("ul").addEventListener("click", function(e) {
  if (e.target.tagName === "LI") {
    console.log("Clicked:", e.target.textContent);
  }
});
```

ES6+ Features

Q139. ES6 kya hai aur ismein kya naya aaya?

ES6 (ECMAScript 2015) JavaScript ka major update tha jisme let/const, arrow functions, template literals, classes, destructuring, promises, modules jaise features add hue.

Q140. JavaScript classes kaise kaam karte hain?

Classes objects create karne ka blueprint provide karte hain, constructor function aur methods ke sath — yeh syntactic sugar hai prototype-based inheritance ke upar.

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return `Hi, I am ${this.name}`;
  }
}
let p = new Person("Nik");
console.log(p.greet());
```

Q141. JavaScript me class inheritance kaise implement karte hain?

extends keyword se ek class doosri class ko inherit karti hai, aur super() call parent constructor ko invoke karta hai.

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
}
class Dog extends Animal {
  speak() { return `${this.name} barks`; }
}
```

Q142. let/const block scoping var se kaise different hai?

var function-scoped hota hai (block ignore karta hai) jisse bugs ho sakte hain loops me. let/const block-scoped hote hain, matlab {} ke andar hi accessible rehte hain, jo predictable behavior deta hai.

```
if (true) {
  var x = 1;
  let y = 2;
}
console.log(x); // 1
console.log(y); // Error: y is not defined
```

Q143. JavaScript modules (import/export) kya hote hain?

Modules code ko separate files me organize karne dete hain. export keyword se functions/variables share kiye jaate hain aur import se doosri file me use kiye jaate hain — code reusability aur maintainability improve hoti hai.

```
// math.js
export function add(a, b) { return a + b; }

// app.js
import { add } from "./math.js";
console.log(add(2, 3));
```

Q144. Optional chaining (?.) aur nullish coalescing (??) kya karte hain?

Optional chaining (?.) safely nested object properties access karta hai bina error diye agar koi property undefined/null ho. Nullish coalescing (??) default value deta hai sirf jab left side null/undefined ho (0 ya "" false nahi maane jate, unlike ||).

```
let user = {};
console.log(user?.address?.city); // undefined (no error)

let count = 0;
console.log(count ?? 10); // 0 (not 10, because 0 is not null/undefined)
```

Async JavaScript / Promises

Q145. Synchronous aur Asynchronous code me kya difference hai?

Synchronous code line-by-line, ek time par ek hi task execute karta hai (blocking). Asynchronous code background me chalta hai (jaise API calls, timers) bina baaki code ko block kiye — JS single-threaded hone ke bawajood non-blocking behave kar sakti hai event loop ki wajah se.

Q146. JavaScript Promise kya hai?

Promise ek object hai jo kisi asynchronous operation ke eventual completion (ya failure) ko represent karta hai. Yeh 3 states me hota hai: pending, fulfilled, ya rejected.

```
let promise = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Done!"), 1000);
});
promise.then(result => console.log(result));
```

Q147. async/await kaise kaam karta hai?

async/await Promises ke sath kaam karne ka cleaner syntax hai. async function hamesha promise return karti hai, aur await keyword promise ke resolve hone tak function execution ko pause kar deta hai (bina thread block kiye).

```
async function fetchData() {
  let response = await fetch("api/data");
  let data = await response.json();
  console.log(data);
}
```

Q148. try/catch async code me error handling kaise karta hai?

try block me code likha jata hai jo error throw kar sakta hai, aur catch block us error ko handle karta hai — async/await ke sath yeh Promise rejections ko gracefully handle karne ke liye use hota hai.

```
async function getData() {
  try {
    let res = await fetch("api/data");
    let data = await res.json();
  } catch (error) {
    console.log("Error:", error);
  }
}
```

Q149. Callback function aur callback hell kya hota hai?

Callback ek function hai jo doosre function ko argument ki tarah pass hota hai aur baad me call hota hai. Callback hell tab hota hai jab bahut saare nested callbacks se code deeply indented aur unreadable ho jata hai — Promises/async-await isko solve karte hain.

```
getUser(id, (user) => {
  getPosts(user, (posts) => {
    getComments(posts, (comments) => {
      // deeply nested = callback hell
    });
  });
});
```

Closures, Scope & Advanced Concepts

Q150. JavaScript closure kya hota hai?

Closure ek function hai jo apne outer (lexical) scope ki variables ko yaad rakhta hai, chahe outer function execution complete ho chuka ho. Isse data privacy aur state preserve karne me help milti hai.

```
function counter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}
let increment = counter();
console.log(increment()); // 1
console.log(increment()); // 2
```

Q151. Hoisting kya hai?

Hoisting JavaScript ka behavior hai jisme variable aur function declarations execution se pehle unke scope ke top par 'move' ho jati hain (memory me allocate hoti hain). var undefined ke sath hoisted hota hai, let/const 'temporal dead zone' me rehte hain.

```
console.log(x); // undefined (hoisted)
var x = 5;

console.log(y); // ReferenceError
let y = 5;
```

Q152. 'this' keyword ka value kaise determine hota hai?

'this' ka value function **kaise call hua** uspar depend karta hai, na ki kaha define hua. Regular function me object call karta hai to 'this' woh object hota hai; global scope me 'this' global object hota hai; arrow functions apna 'this' nahi rakhte, enclosing scope ka use karte hain.

```
const obj = {
  name: "Nik",
  greet() { console.log(this.name); }
};
obj.greet(); // "Nik"
```

Q153. call(), apply() aur bind() me kya difference hai?

Teeno function ke 'this' context set karne ke liye use hote hain. **call()** arguments ko comma se individually pass karke function turant invoke karta hai. **apply()** same hai but arguments array form me leta hai. **bind()** naya function return karta hai with fixed 'this', turant invoke nahi karta.

```
function greet(greeting) { console.log(`${greeting}, ${this.name}`); }
const person = { name: "Nik" };
greet.call(person, "Hi");
greet.apply(person, ["Hello"]);
const bound = greet.bind(person);
bound("Hey");
```

Q154. Event loop kya hai aur JavaScript single-threaded hone ke bawajood async kaam kaise karti hai?

JavaScript ek single-threaded language hai (ek time par ek task), lekin browser APIs (setTimeout, fetch) background me kaam karte hain. Event loop call stack ko monitor karta hai aur jab woh empty hoti hai, tab callback/microtask queue se pending tasks ko execute karne ke liye push karta hai — isse asynchronous behavior possible hoti hai.